DAW 4 – DISEÑO RESPONSIVO

DISEÑO RESPONSIVO

- El LAYOUT de una página es la forma en la que se disponen sus distintos elementos.
  - En los **primeros años de la web**, los diseños de layout se hacían para un **tamaño de pantalla fijo**, por lo que al abrir la página con otros tamaños aparecían barras de desplazamiento o espacios desaprovechados.
    - Las dos opciones tradicionales para abordar el diseño del layout eran:
      - Layout **fijo** → se fija de forma absoluta el tamaño de las cajas, con independencia del tamaño de la ventana (hay que fijar el ancho del contenido total).
      - Layout **líquido** → no se fija un tamaño fijo, el navegador adapta el tamaño de los diferentes elementos al tamaño de la ventana y la cantidad de contenidos.
  - La **aproximación actual** es el **mobile first design**, que consiste crear un diseño simple (a una sola columna), pensado para móviles y luego comprobar si se va a abrir con pantallas más grandes, en cuyo caso se aplicará un diseño a varias columnas.
    - Las opciones actuales para abordar el diseño del layout son resultado de una evolución:
      - **Cameron Adams (2004)** introduce la idea de crear diseños que se adapten al tamaño de la pantalla, usando JavaScript para detectar dicho tamaño y cargar así la versión CSS adecuada.
      - **Ethan Marcotte (2010)** introduce el término "diseño responsivo" basado en rejillas fluidas, imágenes fluidas y solicitudes de tipo de medio.

- El Diseño Web Responsivo (RWD) son las prácticas que se usan para que una página web se adapte automáticamente al tamaño del dispositivo.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- El VIEWPORT es el área de una página web visible para el usuario.

↳ Establece el ancho de la página al del dispositivo y establece el zoom inicial a 1.

REGLAS @media

- Las REGLAS @media se aplican sólo si se cumple una condición sobre la resolución del dispositivo.
  @media not/only *mediatype* and (*mediafeature* and/or/not *mediafeature*) { CSS-Code; }
  - Los **tipos de dispositivo** (*mediatype*) pueden ser:
    - all → todos los tipos (default).
    - print → impresoras.
    - screen → pantallas de ordenador, tablet, móvil, etc.
  - Las **características de los dispositivos** (*mediafeature*) que se pueden chequear son, entre otras:
    - width y height → ancho y alto del viewport.
    - orientation → orientación (paisaje o retrato) del viewport.
    - resolution → resolución del dispositivo..
  - not → revierte el significado de lo que viene a continuación.
  - only → previene a los navegadores que no soportan las reglas @media (no tiene efecto en navegadores modernos).
  - and → permite combinar tipos de dispositivos (*mediatype*) y sus características (*mediafeature*).

```
/* Aplicar esta regla para anchuras menores a 600px */
@media only screen and (max-width: 600px) {
    body {
        background-color: black;
        color: white;
        width: 300px;
    }
}
```

- Los PUNTOS DE RUPTURA son las dimensiones a partir de las cuales se planifica el cambio de diseño del layout. Tamaños típicos:

min-width: 320 px	Móviles de pequeño tamaño.
min-width: 480 px	Pequeños dispositivos y la mayoría de móviles.
min-width: 768 px	La mayoría de tablets.
min-width: 992 px	Ordenadores con pantallas pequeñas.
min-width: 1200 px	Grandes dispositivos y ordenadores con pantallas grandes.

❖ Alternativamente se podrían incluir estas reglas como parte de las etiquetas **<link>** que permiten seleccionar **hojas de estilo alternativas**.

```
<link … media="screen and (min-width: 900px)" href="pantallaGrande.css">

<link … media="screen and (max-width: 600px)" href="pamtallaPequeña.css">
```

TIPOGRAFÍA E IMÁGENES RESPONSIVAS

- **Tipografía responsiva** → tamaños de fuente adaptados al del dispositivo.

```
<body>
    <h1>Tipografía Responsiva</h1>
    <p>Este es un ejemplo de tipografía responsiva</p>
</body>
```

Tipografía Responsiva

Este es un ejemplo de tipografía responsiva

```
html {
  font-size: 12px;
}
h1 {
  font-size: 2rem;
}
@media (min-width: 650px) {
  h1 {
    font-size: 4rem;
  }
}
p {
  font-size: calc(1rem + 1vw);
}
```

Tipografía Responsiva

Este es un ejemplo de tipografía responsiva

- **Imágenes responsivas** → imágenes adaptadas en origen al tamaño del dispositivo.
  - Solución tradicional:

```
img {
    max-width: 80%;
}
```

  - Solución actual:

```
<picture>
  <source media="(min-width:900px)" srcset="miImagen_1.jpg">
  <source media="(min-width:650px)" srcset="miImagen_2.jpg">
  <img src="miImagen_3.jpg">
</picture>
```

FLEXIBLE GRIDS

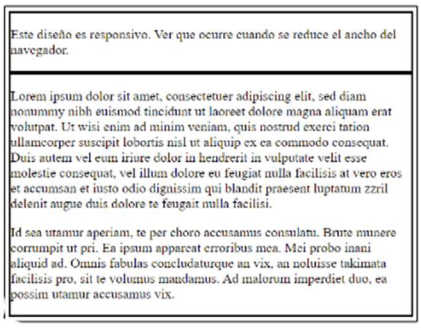
- FLEXIBLE GRIDS es el mecanismo anticuado de responsividad basado en:
  - Usar **reglas** $@\mathbf{media}$ para añadir atributos en dispositivos grandes o pequeños.
  - Usar la propiedad $\mathbf{float}$ para mover elementos.
  - Abusar **de tamaños relativos** (con %) en los elementos → $\text{width} = 100 * \text{anchodeseado}/\text{anchopagina}$.

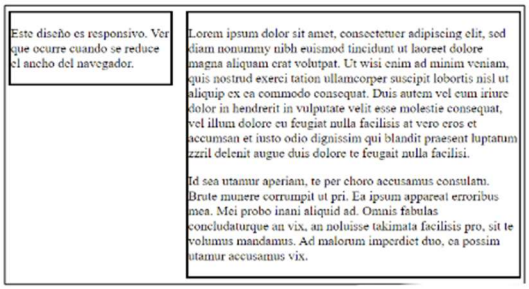
```html
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
</head>

<body>

    <div class="contenedor">
      <div class="col1">
        <p>Este diseño es responsivo. Ver que ocurre cuando se reduce el
      </div>
      <div class="col2">
          <p>Lorem ipsum dolor sit amet, consectetuer adipiscing elit, s
          <p>Id sea utamur aperiam, te per choro accusamus consulatu. Br
      </div>
    </div>
</body>
```

```css
@media screen and (min-width: 600px) {
  .col1 {
    width: 31.25%;
    float: left;
  }
  .col2 {
    width: 64.75%;
    float: right;
  }
}
```





TECNOLOGÍAS DE LAYOUT MODERNAS

- Las **tecnologías de layout modernas** son métodos que asumen que el diseño que se va a crear es flexible y facilitan su implementación.

MULTICOL

- MULTICOL es el método más antiguo, se basa en indicarle al navegador en cuántas **columnas** se quiere **dividir un contenedor** y que él haga los cálculos en función del **tamaño del viewport**.
  - El número de columnas se especifica a través de la propiedad column-count.

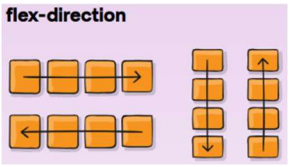
FLEX CONTAINER

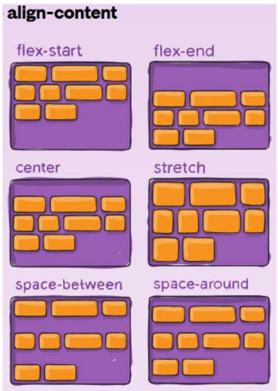
- FLEX CONTAINER es un método diseñado crear un **contenedor flexible** que facilita la **disposición de elementos** en **una dimensión**, con filas **o** columnas.

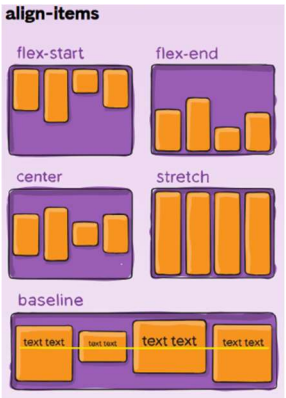
Una flexbox está compuesta por:

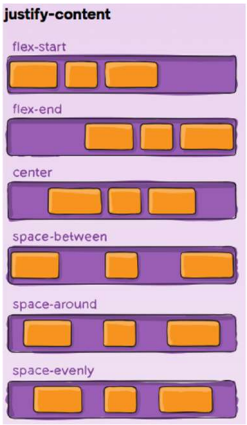
- Un **contenedor flex** → contenedor con la propiedad display: flex.

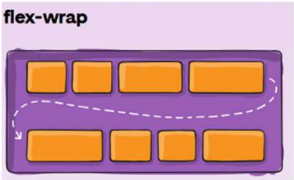
○ flex-direction → dirección de display de los ítems. ▪ $\text{row, column, row-reverse, column-reverse}$.	○ align-content → alinear las líneas flex cuando los ítems wrapean a varias líneas. ▪ $\text{stretch, center, space-between, space-around, space-evenly}$.
○ flex-wrap → especifica si los ítems deben hacer wrap o no cuando no hay espacio suficiente en una línea flex. ▪ $\text{nowrap, wrap, wrap-reverse}$.	○ justify-content → alinea los ítems cuando no usan todo el espacio disponible en el eje principal (horizontalmente). ▪ $\text{flex-start, flex-end, center, space-between, space-around, space-evenly}$
○ flex-flow → propiedad resumen de las 2 anteriores.	○ place-content → propiedad resumen de las 2 anteriores
○ align-items → alinea los ítems cuando no usan todo el espacio disponible en el eje no principal (verticalmente). ▪ $\text{flex-start, flex-end, stretch, baseline}$.	○ gap → propiedad resumen de row-gap y column-gap.





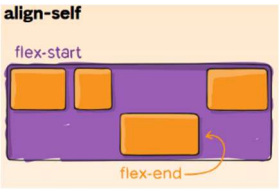






- Uno o más **ítems flex** → hijos directos del contenedor flex.

○ flex-grow → el ítem crecerá X veces más rápido.	○ order → especifica el orden de display de los ítems dentro del contenedor.
○ flex-shrink → el ítem decrecerá X veces más rápido.	○ align-self → especifica la alineación del ítem dentro del contedor. ↳ Sobrescribe la alineación impuesta por la propiedad align-items del contenedor.
○ flex-basis → longitud inicial de un ítem.	
○ flex → propiedad resumen de las 3 anteriores.	



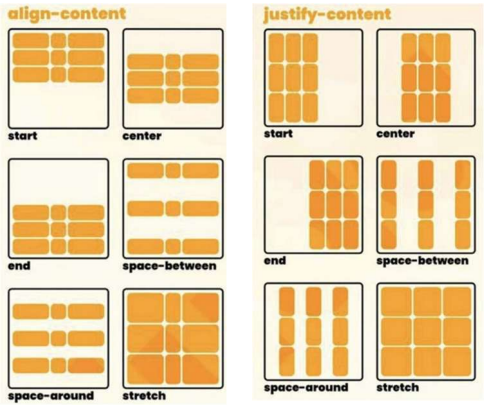
GRID

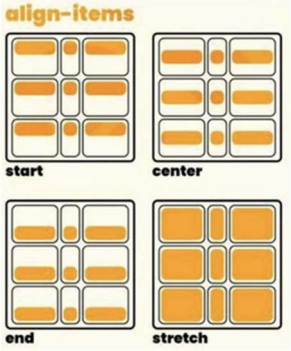
- GRID es un método diseñado para crear un **contenedor flexible** que facilita la **disposición de elementos** en **dos dimensiones**, tanto en **filas** como en **columnas**.

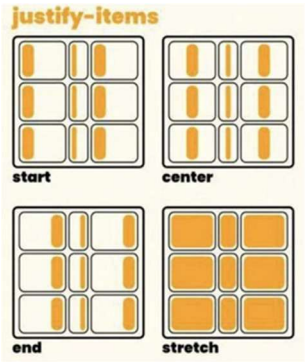
Una grid está compuesta por:

- Un **contenedor grid** → contenedor con la propiedad display: grid.

○ grid-template-rows → tamaño de las columnas.	○ align-items → alineamiento vertical por defecto de los ítems en las celdas. ▪ stretch, start, end, center, baseline.
○ grid-template-columns → tamaño y número de columnas • Permite tamaños fijos (px), relativos (%), automáticos (auto) o con la unidad fr (fraction), que divide el espacio de la tabla en fracciones.	○ justify-items → alineamiento horizontal por defecto de los ítems en las celdas. ▪ stretch, start, end, center.
○ grid-template-areas → permite referenciar los ítems con nombre para especificar las filas que muestran. ↳ Cada fila va encerrada entre "". ↳ Un . indica una celda vacía.	○ align-content → alinea verticalmente la tabla en el contenedor. ▪ start, end, center, stretch, space-around, space-between, space-evenly.
○ grid-template → propiedad resumen de las 3 anteriores.	○ justify-content → alinea verticalmente los ítems dentro del contenedor. ▪ center, start, end, space-around, space-between, space-evenly
	○ gap → propiedad resumen de row-gap y column-gap.







- Uno o más **ítems grid** → hijos directos del contenedor grid.

○ grid-column-start → columna en la que empieza el ítem.	○ grid-area → permite ponerle nombre al ítem.
○ grid-column-end → columna en la que termina el ítem.	○ justify-self → alineamiento horizontal del ítem. ↳ Sobrescribe la alineación impuesta por la propiedad justify-items del contenedor.
○ grid-column → propiedad resumen de las 2 anteriores.	○ align-self → alineamiento vertical del ítem. ↳ Sobrescribe la alineación impuesta por la propiedad align-items del contenedor.
○ grid-row-start → fila en la que empieza el ítem.	
○ grid-row-end → fila en la que termina el ítem.	
○ grid-row → propiedad resumen de las 2 anteriores.	

```html
<div class="contenedor">
    <div class="box1">Uno</div>
    <div class="box2">Dos</div>
    <div class="box3">Tres</div>
</div>
```

HTML

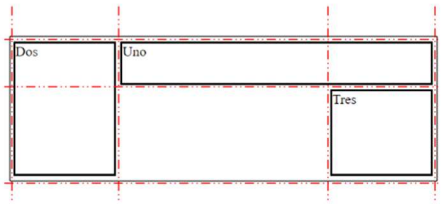
```css
.contenedor {
    display: grid;
    grid-template-columns: 1fr 2fr 1fr;
    grid-template-rows: 50px 100px;
    grid-gap: 5px;
}

.box1 {
    grid-column: 2 / 4;
    grid-row: 1;
}

.box2 {
    grid-column: 1;
    grid-row: 1 / 3;
}

.box3 {
    grid-column: 3;
    grid-row: 2;
}
```

CSS



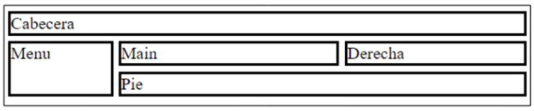

```html
<div class="contenedor">
  <div class="item1">Cabecera</div>
  <div class="item2">Menu</div>
  <div class="item3">Main</div>
  <div class="item4">Derecha</div>
  <div class="item5">Pie</div>
</div>
```

HTML

```css
.item1 { grid-area: header; }
.item2 { grid-area: menu; }
.item3 { grid-area: main; }
.item4 { grid-area: right; }
.item5 { grid-area: footer; }

.contenedor {
  display: grid;
  grid-template-areas:
    'header header header header header header'
    'menu main main main right right'
    'menu footer footer footer footer footer';
  grid-gap: 5px;
}
```

CSS



FRAMEWORKS CSS

- ○ Los FRAMEWORKS son bibliotecas de **reglas** CSS **predefinidas** apoyadas en el atributo **class** de HTML.
- ✓ Facilitan la **compatibilidad** entre navegadores y **responsividad** entre dispositivos.
- ✓ **Simplifican** el desarrollo de la aplicación web.
- ✓ Garantizan cierto grado de **fiabilidad** y **eficacia**.
- × Se importa **código innecesario**, incrementando **ancho de banda**.
- × Se **pierde** cierto **control** sobre lo que se está haciendo.
- × Se **limitan** las posibilidades de **elección del diseño**.

BOOTSTRAP

- BOOTSTRAP es una librería diseñada para el RWD, **especialmente para móviles**, que se activa añadiendo el siguiente código html:

```
<link rel="stylesheet" href="bootstrap.css">
<script src="bootstrap.js"></script>
```

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist
/css/bootstrap.min.css"">

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist
/js/bootstrap.bundle.min.js"></script>
```

- Fue creado por Mark Otto y Jacob Thronton en Twitter y liberado en 2011 en GitHub.

INTEGRACIÓN CON HTML

- Existen **diversas versiones**, pero en la actualidad se usa la **5** (2021).
- ✓ Bootstrap 5 es la versión **más nueva**, con nuevos componentes, hoja de estilos más rápida y más capacidad de respuesta.
- ✓ Bootstrap 5 es **compatible** con las últimas versiones estables de los **principales navegadores** y **plataformas** (no es compatible con Internet Explorer 11 y anteriores).
- ✓ Bootstrap 5 ha cambiado a **JavaScript nativo** en lugar de usar jQuery.

CONTENEDORES

Bootstrap ofrece 3 clases fundamentales para crear contenedores:

- container → crea un contenedor que adapta su max-width en **cada punto de ruptura** prefijado.
- container-XX → crea un contenedor que adapta su max-width en **cada punto de ruptura** prefijado, dependiendo del **sufijo XX** (sm, md, lg, xl, xxl).
- container-fluid → crea un contenedor líquido con max-width establecido a **100%** **para cualquier punto de ruptura** prefijado.

min-width donde se produce el cambio (punto de ruptura)

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra Large ≥1200px	XXL ≥1400px
max-width	100%	540px	720px	960px	1140px	1320px

max-width que toma el contenedor

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
.container	100%	540px	720px	960px	1140px	1320px
.container-sm	100%	540px	720px	960px	1140px	1320px
.container-md	100%	100%	720px	960px	1140px	1320px
.container-lg	100%	100%	100%	960px	1140px	1320px
.container-xl	100%	100%	100%	100%	1140px	1320px
.container-xxl	100%	100%	100%	100%	100%	1320px
.container-fluid	100%	100%	100%	100%	100%	100%

ESPACIADOS

- Bootstrap permite dar **espacios** asignando **clases** de la forma
  {propiedad}{lados} − {tamaño} (para pantallas xs) / {propiedad}{lados} − {punto-ruptura}{tamaño} (para pantallas sm, md, … xxl).

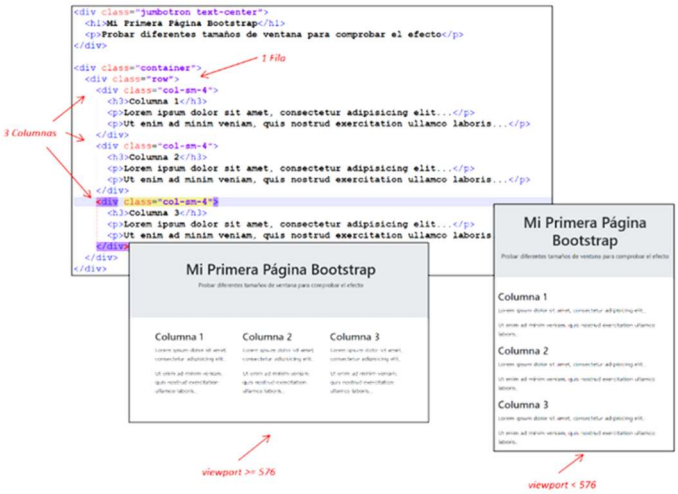
Donde propiedad es	Donde lado es:	Donde punto-ruptura es:	Donde tamaño es:
▪ m → margin. ▪ p → padding.	▪ t → top. ▪ b → bottom. ▪ l → left. ▪ r → right. ▪ x → fija l y r simultáneamente. ▪ y → fija t y b simultáneamente.	• sm. • md. • lg. • xl. • xxl.	▪ 0 → para clases que eliminan el margen o el relleno estableciéndolo en 0. ▪ 1 → para clases que establecen el margen o relleno a $spacer ∗ 0.25. ▪ 2 → para clases que establecen el margen o relleno a $spacer ∗ 0.5. ▪ 3 → para clases que establecen el margen o relleno a $spacer. ▪ 4 → para clases que establecen el margen o relleno a $spacer ∗ 1.5. ▪ 5 → para clases que establecen el margen o relleno a $spacer ∗ 3. ↳ $spacer tiene un valor por defecto, pero es configurable. ▪ auto → para las clases que establecen el margen en auto.

LAYOUT

- El mecanismo de creación de **layout** de Bootstrap está basado en el uso de **flexbox** para la implementación de **rejillas de hasta 12 columnas** que se pueden reagrupar para crear columnas más anchas.

Los pasos a seguir para usar esta rejilla son:

1. Crear un **contenedor** con las clases container o container-fluid.
2. Dentro del contenedor, crear tantas **filas** como se desee usando la clase row.
3. Dentro de cada fila, especificar el **número de columnas** usando la estructura de clase col-{dispositivo}-{numero de columnas}.



Sufijo	Dispositivo	Punto de Ruptura	max-width
-	Extra Small	< 576 px	-
sm	Small	>= 576 px	540px
md	Medium	>= 768 px	720px
lg	Large	>= 992 px	960px
xl	Extra Large	>= 1200 px	1140px
xxl	Extra Extra Large	>= 1400 px	1320px

PROPIEDADES FIJADAS POR BOOTSTRAP

- **Texto/tipografía** → rem = 16px, $spacer = 1rem, line-height=1.5rem.
- **Tablas** → incluye diferentes formatos de presentación.
- **Imágenes** → incluye posibilidad de redondeo de esquinas e inclusión de marcos.
- **Párrafos resaltados** → mediante la clase jumbotron.
- **Redondeo de los bordes** de las cajas → mediante la clase well.
- Inclusión de **iconos acompañando texto** → mediante la clase glyphicon.
- Inclusión de **paginación y migas de pan** → mediante las clases pagination y breadcrumb.
- Inclusión de **desplegables** → mediante la clase dropdown.
- Inclusión de **barras de navegación** → mediante las clases navbar y navbar-*
- Inclusión de **carruseles de imágenes** → mediante la clase carousel-*